



E-TOOLS AI CORPORATION

Abe Pro

User Manual

Level 1: General Programming • Runtime Library

Part III: The Runtime Library

Compiler: abec 1.0.0 (v4.1 grammar) · Linux x86-64 · Revised 2026-07-02

Contents

Overview.....	3
1. Getting Started.....	3
2. Types and Variables.....	4
3. Functions.....	7
4. Control Flow.....	9
5. Collections.....	12
6. Error Handling.....	15
7. Classes and Enums.....	16
8. Modules.....	18
Part III — The Runtime Library.....	20
9. Strings, JSON, and `typeof`.....	20
10. Crypto and Authentication.....	22
11. Environment, Files, and Caching.....	23
12. HTTP.....	25
13. Database.....	26
14. Licensing.....	27
Appendix A — A Worked Example.....	28

Overview

Abe is a typed, memory-safe language with familiar TypeScript-flavored syntax that compiles **straight to native code** through LLVM. The compiler, **abec**, turns a **.abe** source file into an ordinary executable — no virtual machine, no interpreter, no **node_modules**. Programs link a compact **runtime library** that ships, batteries-included, with strings and JSON, collections, crypto and auth, an HTTP server and client, caching, files, and a PostgreSQL client — so a complete web service is a single self-contained binary.

This manual is organized in three parts plus a worked example:

- **Part I — Getting Started:** install **abec**, compile your first program, and the basics of the toolchain.
- **Part II — The Level 1 Language:** types, functions, control flow, collections, error handling, classes/enums, and modules.
- **Part III — The Runtime Library:** the batteries — strings/JSON, crypto & auth, environment/files/caching, HTTP, and the database client.
- **Appendix A — A Worked Example:** the pieces combined into a small live JSON API.

New to Abe? Read Part I, skim Part II, and reach for Part III as you build. Coming from TypeScript? The syntax will feel familiar; watch the "not yet" callouts for the handful of features still landing.

Scope of this release. This manual covers **Level 1** (general-purpose programming) and the **runtime library** you can call from Abe source. Every code example here is compiled and run by the regression suite (**tests/run_manual_examples.sh**) against the exact **abec** you download — if an example appears in this manual, it compiles and produces the output shown. **Level 2 (Machine Learning)** — tensors, layers, models, training, generation, autodiff, tokenizers, LoRA — ships in a **later release** and is documented separately. Where a Level-1 feature is specified but not yet lowered by the compiler, this manual says so explicitly rather than showing an example that would not compile.

1. Getting Started

1.1 What is Abe?

Abe is a typed, memory-safe systems language that compiles **.abe** source to native code through LLVM. **abec** is the compiler. Programs are ordinary native executables — no virtual machine, no runtime interpreter — and link a small runtime library that provides strings, collections, JSON, crypto, HTTP, and more (Part III).

1.2 Installing `abec`

Download the prebuilt Linux x86-64 compiler and the runtime it links. Keep them side by side — **abec** finds an adjacent **runtime/** automatically:

```
# The compiler (a single self-contained binary):
curl -fsSL -o abec \
  https://github.com/E-Tools-AI-Corporation/abec/releases/latest/download/abec-
linux-x86_64
chmod +x abec
./abec --version          # -> abec 1.0.0 (v4.1 grammar)

# The runtime (needed to link native programs):
curl -fsSL -O \
```

```
https://github.com/E-Tools-AI-Corporation/abec/releases/latest/download/abe-
runtime-1.0.0-linux-x86_64.tar.gz
tar xzf abe-runtime-1.0.0-linux-x86_64.tar.gz # creates ./runtime next to abec
```

Building a native executable runs the LLVM toolchain, so **clang** and **llc** must be on your **PATH** (install your distribution's **clang** / **llvm** packages). **abec** runs free for a **30-day evaluation**; after that it needs a license — see Chapter 14.

If **abec** and **runtime/** live in different places, point the compiler at the runtime with **export ABE_RUNTIME_DIR=/path/to/runtime**.

1.3 Your first program

Create **hello.abe**:

```
function main(): i64 {
    console.log("Hello, Abe!");
    return 0;
}
```

Compile and run it:

```
./abec hello.abe -o hello # compile to a native binary
./hello # prints: Hello, Abe!
```

1.4 The execution model — `main()` is required

Every Abe program starts at **main**, which **must** be declared as **function main(): i64**. Its return value is the process exit code (0 means success). Unlike scripting languages, top-level statements do **not** execute on their own — put your logic inside **main** (or functions it calls).

1.5 The `abec` toolchain

Command	What it does
<code>abec FILE -o OUT</code>	Compile FILE to the native executable OUT
<code>abec --check FILE</code>	Parse + type-check only (no code generated)
<code>abec --emit-ll FILE</code>	Print the generated LLVM IR to stdout
<code>abec --version</code>	Print the compiler version

`console.log(...)` prints its arguments separated by spaces, followed by a newline. It accepts strings, numbers, and booleans.

2. Types and Variables

2.1 Declaring variables

Abe has three binding forms. **let** and **var** are mutable; **const** is immutable. A type annotation (**: i64**) is optional when the type can be inferred from the initializer.

```
function main(): i64 {
    let count: i64 = 3; // mutable, annotated
    const name = "Abe"; // immutable, inferred
```

```

    var total = count + 1;    // mutable, inferred
    if (total !== 4) return 1;
    if (name !== "Abe") return 2;
    return 0;
}

```

Variables also work at **module scope** (top level): a `const` for shared configuration, or a `let` for module-wide mutable state that functions read and update. See §2.7.

2.2 Numeric types

The core numeric types are `i64` (64-bit signed integer) and `f64` (64-bit float). Arithmetic uses the usual precedence — `*`, `/`, `%` bind tighter than `+`, `-`.

```

function main(): i64 {
    let a: i64 = 3 + 4 * 2;    // 11 — '*' binds tighter than '+'
    let rem: i64 = 10 % 3;     // 1 — remainder
    let pow: i64 = 2 ** 10;    // 1024 — exponent operator
    console.log(a, rem, pow); // prints: 11 1 1024
    return 0;
}

```

The `**` operator also works on floats (e.g. `9.0 ** 0.5` is `3.0`).

Bitwise and shift operators work on integers:

```

function main(): i64 {
    if ((6 & 3) !== 2) return 1; // AND
    if ((6 | 1) !== 7) return 2; // OR
    if ((6 ^ 3) !== 5) return 3; // XOR
    if ((1 << 4) !== 16) return 4; // shift left
    if ((32 >> 2) !== 8) return 5; // shift right
    return 0;
}

```

Floating-point values use `f64`:

```

function main(): i64 {
    let price: f64 = 2.5;
    let total: f64 = price * 4.0;
    if (total !== 10.0) return 1;
    return 0;
}

```

2.3 Strings and template literals

Strings concatenate with `+`. Template literals (backticks) interpolate expressions with `${...}`:

```

function main(): i64 {
    let who: string = "Abe";
    let version: i64 = 4;
    let greeting = `Hello, ${who} — v${version}!`; // "Hello, Abe — v4!"
    console.log(greeting);
    let joined = "a" + "b" + "c";
    if (joined !== "abc") return 1;
    return 0;
}

```

}

2.4 Arrays

Array literals are written with `[...]`. Read the length with `.length`, index with `[i]`, and iterate with `for ... of`:

```
function main(): i64 {
    let nums = [10, 20, 30];
    if (nums.length != 3) return 1;
    if (nums[1] != 20) return 2;
    let sum = 0;
    for (const n of nums) { sum = sum + n; }
    if (sum != 60) return 3;
    return 0;
}
```

Arrays of numbers, strings, and booleans all work; functional methods (`.map`, `.filter`, `.forEach`) are covered in the Collections chapter.

2.5 Objects

Object literals map string keys to values and are read with `.field`:

```
function main(): i64 {
    let user = { name: "Alice", role: "admin" };
    if (user.name != "Alice") return 1;
    if (user.role != "admin") return 2;
    return 0;
}
```

In this release object-literal members hold **string** values (the form used by JSON and configuration). Numeric-valued members are a documented limitation and are slated for a future release.

2.6 Null safety

The nullish-coalescing operator `??` yields its left side unless that side is null, in which case it yields the right side. The ternary operator `?:` chooses between two values:

```
function pick(value: string): string {
    return value ?? "default"; // nullish coalescing
}
function main(): i64 {
    let label = 7 > 3 ? "big" : "small"; // ternary
    if (label != "big") return 1;
    if (pick("set") != "set") return 2;
    return 0;
}
```

2.7 Module-scope variables

Declarations at the top level of a file (outside any function) are visible to every function in the module: a `const` for shared configuration, or a `let` for module-wide mutable state.

```
const MAX_RETRIES: i64 = 3; // module-level const
let requestCount: i64 = 0; // module-level mutable state
```

```
function record(): void { requestCount = requestCount + 1; }

function main(): i64 {
    record();
    record();
    if (MAX_RETRIES != 3) return 1;
    if (requestCount != 2) return 2;
    return 0;
}
```

Functions may be declared in any order relative to the module-level variables they use — initialization runs before `main`.

3. Functions

3.1 Declaring functions

A function declares typed parameters and a return type. The body returns a value of that type (or nothing, for `void`).

```
function add(a: i64, b: i64): i64 { // typed params + return type
    return a + b;
}
function greet(name: string): string {
    return "hi " + name;
}
function main(): i64 {
    if (add(2, 3) != 5) return 1;
    if (greet("Abe") != "hi Abe") return 2;
    return 0;
}
```

A `void` function returns no value — it runs for its side effects:

```
function announce(msg: string): void { // void: returns no value
    console.log(msg);
}
function main(): i64 {
    announce("ready"); // prints: ready
    return 0;
}
```

3.2 Default parameters

A parameter may declare a default value, used when the caller omits that argument. (Defaults apply from the trailing end of the parameter list.)

```
function greet(name: string = "world"): string {
    return "hi " + name;
}
function main(): i64 {
```

```

    if (greet() != "hi world") return 1;    // default used
    if (greet("Abe") != "hi Abe") return 2; // overridden
    return 0;
}

```

3.3 Recursion

Functions may call themselves:

```

function fib(n: i64): i64 {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
function main(): i64 {
    if (fib(10) != 55) return 1;
    return 0;
}

```

3.4 Arrow functions and first-class function values

An **arrow function** (`params`) `=> expr` is a function value. It can be stored in a variable and called, passed as an argument, or returned:

```

function main(): i64 {
    let inc = (x: i64): i64 => x + 1;
    let add = (a: i64, b: i64): i64 => a + b;
    if (inc(9) != 10) return 1;
    if (add(20, 22) != 42) return 2;
    return 0;
}

```

Arrow functions are also the natural argument to the collection methods `.map`, `.filter`, and `.forEach` (see the Collections chapter).

3.5 Higher-order functions

A function can take another function as a parameter, typed with a **function type** `(name: T) => U`. Both named functions and arrows can be passed:

```

function apply(f: (n: i64) => i64, x: i64): i64 {
    return f(x);
}
function double(n: i64): i64 { return n * 2; }
function main(): i64 {
    if (apply(double, 21) != 42) return 1;           // pass a named function
    if (apply((n: i64): i64 => n + 1, 9) != 10) return 2; // pass an arrow
    return 0;
}

```

Function-type annotations require **named** parameters — write `(n: i64) => i64`, not `(i64) => i64`.

3.6 Generic functions

A function can be parameterized by a type with `<T>`. Generics are **pass-through**: a `T` value is carried unchanged from input to output. This covers identity and forwarding helpers:


```
function identity<T>(x: T): T { return x; }
function main(): i64 {
  let n: i64 = identity(5);
  let s: string = identity("abe");
  if (n != 5) return 1;
  if (s != "abe") return 2;
  return 0;
}
```

The concrete type is inferred from the argument. Annotate the result (`let n: i64 = ...`) so the value is recovered at its real type.

Limit: a `T` value is opaque — you can pass it through, store it, and return it, but you cannot do arithmetic or comparison on it inside the generic function. Explicit call-site type arguments (`identity<i64>(5)`) are not yet supported; rely on inference.

3.7 Async functions

`async` functions and `await` are accepted, but the runtime is **single-threaded and runs them synchronously** — `await e` simply evaluates `e`. There is no event loop.

```
async function compute(x: i64): i64 {
  return x * 2;
}
function main(): i64 {
  let r = await compute(21); // runs synchronously
  if (r != 42) return 1;
  return 0;
}
```

For real concurrency, use **actors** and **task groups** (a separate topic), which provide `Sendable`-checked message passing rather than an async event loop.

4. Control Flow

4.1 If / else

```
function classify(n: i64): string {
  if (n < 0) {
    return "negative";
  } else if (n == 0) {
    return "zero";
  } else {
    return "positive";
  }
}
```

4.2 Loops

A `while` loop runs while its condition holds:

```
function main(): i64 {
  let i = 0;
```

```

    let sum = 0;
    while (i < 5) {
        sum = sum + i;
        i = i + 1;
    }
    if (sum != 10) return 1;    // 0+1+2+3+4
    return 0;
}

```

A **do ... while** loop runs the body at least once before testing:

```

let n = 0;
do {
    n = n + 1;
} while (n < 3);

```

A C-style **for** loop has init, condition, and step clauses:

```

let product = 1;
for (let i = 1; i <= 5; i = i + 1) {
    product = product * i;    // 5! = 120
}

```

for ... of iterates the elements of an array (numbers, strings, or booleans):

```

function main(): i64 {
    let total = 0;
    for (const n of [10, 20, 30]) { total = total + n; }    // 60
    let joined = "";
    for (const word of ["a", "b", "c"]) { joined = joined + word; }    // "abc"
    if (total != 60 || joined != "abc") return 1;
    return 0;
}

```

Use **for ... of** to iterate values. (**for ... in** over indices is not yet lowered — index with a C-style **for** loop when you need positions.)

4.3 break and continue

break exits the nearest loop; **continue** skips to its next iteration:

```

let sum = 0;
for (let i = 0; i < 10; i = i + 1) {
    if (i == 5) break;        // stop the loop
    if (i % 2 == 0) continue; // skip even numbers
    sum = sum + i;            // 1 + 3 = 4
}

```

4.4 switch

switch compares a value against **case** labels (integers or strings). A case runs until **break**; cases without a **break** **fall through** to the next, so stacking case labels shares one body. **default** runs when nothing matches.

```

function describe(n: i64): string {
    let label = "";

```

```

    switch (n) {
      case 1:
        label = "one";
        break;
      case 2:
      case 3:
        label = "few";      // 2 and 3 share this body
        break;
      default:
        label = "many";
    }
    return label;
  }
}

```

4.5 Pattern matching with `match`

`match` is an **expression** — it evaluates to a value. Each arm is `pattern => result`, tried top to bottom. Patterns include literals, ranges (`lo ... hi`), or-patterns (`a | b`), a binding identifier, a wildcard `_`, and an optional `if` guard:

```

function grade(score: i64): string {
  return match score {
    90 ... 100 => "A",          // range
    80 ... 89  => "B",
    0          => "F",          // literal
    s if s < 0 => "invalid",    // binding + guard
    _          => "C"          // wildcard (catch-all)
  };
}

```

`match` also **destructures** arrays and objects, binding parts to names:

```

function main(): i64 {
  let point = [3, 4];
  let r = match point {
    [x, y] => x + y,    // bind elements; also `[first, ...rest]`
    _      => 0
  };
  if (r != 7) return 1;

  let user = { role: "admin" };
  let access = match user {
    { role } => role,    // bind the `role` field
    _      => "none"
  };
  if (access != "admin") return 2;
  return 0;
}

```

Constructor patterns (`Some(x)`) are not yet available — they depend on `enum`/tagged-union support landing in a future release.

5. Collections

5.1 Arrays

Arrays are ordered, growable lists. Create them with `[...]`, read `.length`, index with `[i]`, append with `.push`, and remove the last element with `.pop`:

```
function main(): i64 {
    let xs = [10, 20, 30];
    if (xs.length != 3) return 1;
    if (xs[0] != 10) return 2;
    xs.push(40);           // append -> [10, 20, 30, 40]
    if (xs.length != 4) return 3;
    if (xs[3] != 40) return 4;
    xs.pop();              // remove the last element -> [10, 20, 30]
    if (xs.length != 3) return 5;
    return 0;
}
```

`.pop` also *returns* the element it removed, so you can capture it:

```
function main(): i64 {
    let xs = [10, 20, 30];
    let last = xs.pop();    // last == 30, xs == [10, 20]
    if (last != 30) return 1;
    if (xs.length != 2) return 2;
    return 0;
}
```

Iterate with `for ... of` (see Chapter 4). Arrays of numbers, strings, and booleans are all supported.

5.2 map

`.map` builds a new array by transforming each element with a callback. Its result is itself an array you can index or iterate:

```
function main(): i64 {
    let doubled = [1, 2, 3].map(x => x * 2);    // [2, 4, 6]
    if (doubled.length != 3) return 1;
    if (doubled[2] != 6) return 2;
    return 0;
}
```

5.3 filter

`.filter` keeps the elements for which the callback returns true:

```
function main(): i64 {
    let evens = [1, 2, 3, 4, 5, 6].filter(x => x % 2 == 0);    // [2, 4, 6]
    if (evens.length != 3) return 1;
    let sum = 0;
    for (const n of evens) { sum = sum + n; }    // 12
    if (sum != 12) return 2;
    return 0;
}
```

5.4 forEach

`.forEach` runs a callback for each element, for its side effects. The callback can update variables in the enclosing scope:

```
function main(): i64 {
    let total = 0;
    [10, 20, 30].forEach(n => { total = total + n; });
    if (total != 60) return 1;
    return 0;
}
```

5.5 Chaining

Because `.map` and `.filter` return arrays, they chain:

```
let result = [1, 2, 3, 4, 5, 6]
    .filter(x => x > 2)      // [3, 4, 5, 6]
    .map(x => x * 10);      // [30, 40, 50, 60]
```

5.6 reduce

`.reduce(callback, initial)` threads an accumulator through the array: the callback receives (accumulator, element) and returns the next accumulator. The accumulator's type is that of the `initial` value, and may differ from the element type:

```
function main(): i64 {
    let xs = [1, 2, 3, 4];
    let sum = xs.reduce((acc, x) => acc + x, 0);      // 10
    if (sum != 10) return 1;
    let product = xs.reduce((acc, x) => acc * x, 1);  // 24
    if (product != 24) return 2;
    // Accumulator type differs from the element type:
    let totalLen = ["a", "bb", "ccc"].reduce((acc, w) => acc + w.length, 0);
    if (totalLen != 6) return 3;
    let joined = ["a", "b", "c"].reduce((acc, x) => acc + x, ""); // "abc"
    if (joined != "abc") return 4;
    return 0;
}
```

5.7 find

`.find(predicate)` returns the first element for which the predicate is true, or `null` when none match. Compare the result against `null` before using it:

```
function main(): i64 {
    let users = [{ name: "ada" }, { name: "alan" }, { name: "grace" }];
    let u = users.find(r => r.name == "alan");
    if (u == null) return 1;
    if (u.name != "alan") return 2;
    let missing = users.find(r => r.name == "turing");
    if (missing != null) return 3;      // null: no match
    return 0;
}
```

`.find` is for arrays of objects or strings, where `null` is a sound "not found". For arrays of numbers or booleans — where every value is a valid element and there is no spare "not found" value — use `.indexOf` (which returns `-1` when absent) or `.filter` instead.

5.8 Slicing

`.slice(start, end)` returns a new array with the elements from `start` up to (but not including) `end`. The original array is left unchanged, and the returned elements keep their type — numbers read back as numbers:

```
function main(): i64 {
  let xs = [10, 20, 30, 40, 50];
  let mid = xs.slice(1, 4);    // [20, 30, 40]
  if (mid.length != 3) return 1;
  if (mid[0] != 20) return 2;
  let sum = 0;
  for (const n of mid) { sum = sum + n; }    // 90
  if (sum != 90) return 3;
  return 0;
}
```

5.9 Membership

`.includes(v)` reports whether a value is present, and `.indexOf(v)` returns the index of the first match (or `-1`). Both compare **by value**, and work for arrays of numbers and of strings:

```
function main(): i64 {
  let nums = [10, 20, 30];
  if (!nums.includes(20)) return 1;
  if (nums.indexOf(30) != 2) return 2;
  if (nums.indexOf(99) != -1) return 3;

  let names = ["ada", "alan", "grace"];
  if (!names.includes("alan")) return 4;
  if (names.indexOf("turing") != -1) return 5;
  return 0;
}
```

5.10 Strings ↔ arrays

`.split` turns a string into an array of pieces; `.join` turns an array of strings back into one string. Combined with `.map`, this covers most text processing:

```
function main(): i64 {
  let parts = "a,b,c".split(",");          // ["a", "b", "c"]
  if (parts.length != 3) return 1;
  let trimmed = " x , y ".split(",")
    .map(p => p.trim())
    .join("|");                            // "x|y"
  if (trimmed != "x|y") return 2;
  return 0;
}
```

6. Error Handling

6.1 throw and try / catch

Raise an error with `throw`, and handle it with `try / catch`. The thrown value is bound to the `catch` parameter:

```
function main(): i64 {
  try {
    throw "boom";          // raise an error
    return 1;              // unreachable
  } catch (e) {
    return 0;              // handled
  }
}
```

The caught value is usable — most often a string message:

```
function main(): i64 {
  try {
    throw "access denied";
  } catch (e) {
    console.log(e);        // prints: access denied
  }
  return 0;
}
```

You can throw any value — a string, an object (`throw { code: "E1" }`), etc.

6.2 finally

A `finally` block runs whether or not an error was thrown — use it for cleanup that must always happen:

```
function cleanupCount(doThrow: bool): i64 {
  let cleanups = 0;
  try {
    if (doThrow) { throw "x"; }
  } catch (e) {
    // swallow
  } finally {
    cleanups = cleanups + 1; // runs on BOTH paths
  }
  return cleanups;          // always 1
}
```

6.3 Propagation

A `throw` propagates up the call stack until a `try/catch` handles it — so a deep helper can fail and a caller can recover, without threading error codes through every return:

```
function parseAmount(s: string): i64 {
  if (s == "") { throw "empty input"; }
  return 42;
}

function main(): i64 {
```

```

    try {
      let n = parseAmount(""); // throws inside the callee
      return 1;                // unreachable
    } catch (e) {
      return 0;                // caught here
    }
  }
}

```

Style: for *expected* outcomes (a lookup miss, optional input), prefer returning a value (e.g. ?? with a default, or a sentinel) over `throw`. Reserve `throw` for genuinely exceptional conditions.

7. Classes and Enums

7.1 Classes

A class groups typed fields with a `constructor` and methods. Create an instance with `new`; inside the class, `this` refers to the instance:

```

class Point {
  x: i64;
  y: i64;
  constructor(x: i64, y: i64) {
    this.x = x;
    this.y = y;
  }
  sum(): i64 {
    return this.x + this.y;
  }
}

function main(): i64 {
  let p = new Point(2, 3);
  if (p.x != 2) return 1; // field access
  if (p.sum() != 5) return 2; // method call
  return 0;
}

```

Methods can call other methods on `this`, and `void` methods mutate fields:

```

class Counter {
  n: i64;
  constructor() { this.n = 0; }
  bump(): void { this.n = this.n + 1; }
  twice(): void { this.bump(); this.bump(); }
  value(): i64 { return this.n; }
}

```

7.2 Inheritance

A class may `extends` one base class. The derived class **inherits** the base's fields and methods and can **override** methods:

```

class Animal {

```



```

    speak(): string { return "..."; }
    kind(): string { return "animal"; }
}
class Dog extends Animal {
    speak(): string { return "woof"; } // override
}
function main(): i64 {
    let d = new Dog();
    if (d.speak() != "woof") return 1; // overridden
    if (d.kind() != "animal") return 2; // inherited
    return 0;
}

```

Inherited fields and constructors work too — a derived class with no constructor of its own uses the base's:

```

class Animal {
    name: string;
    constructor(n: string) { this.name = n; }
    describe(): string { return this.name; }
}
class Dog extends Animal {
    bark(): string { return this.name + " barks"; } // uses inherited field
}
function main(): i64 {
    let d = new Dog("Rex");
    if (d.describe() != "Rex") return 1;
    if (d.bark() != "Rex barks") return 2;
    return 0;
}

```

Notes / current limits: the base class must be declared **before** the derived class in the file. `super` calls are not yet supported — a derived constructor initializes inherited fields directly via `this.field`. Single inheritance only; `implements` (interfaces) is type-only.

7.3 Enums

An **enum** defines a set of named integer constants. Values auto-increment from 0, or you can assign them explicitly:

```

enum Status {
    Ok = 200,
    NotFound = 404,
}
function label(s: Status): string {
    if (s == Status.Ok) return "ok";
    if (s == Status.NotFound) return "missing";
    return "other";
}
function main(): i64 {
    if (Status.Ok != 200) return 1;
    if (label(Status.NotFound) != "missing") return 2;
    return 0;
}

```

An enum value is an `i64`, so it works directly in comparisons, `switch`, and `match`. The enum name (e.g. `Status`) is a usable type annotation.

8. Modules

A program can span multiple `.abe` files. A file exposes declarations with `export`, and another file pulls them in with `import { ... } from "./path"` (a relative path, without the `.abe` extension). Compiling the entry file resolves and links the imported modules automatically.

8.1 Exporting and importing functions

`mathlib.abe`:

```
export function square(x: i64): i64 { return x * x; }
export function cube(x: i64): i64 { return x * x * x; }
```

`app.abe`:

```
import { square, cube } from "./mathlib";

function main(): i64 {
    if (square(5) != 25) return 1;
    if (cube(3) != 27) return 2;
    return 0;
}
```

Compile the entry file — the import is resolved for you:

```
./abec app.abe -o app
./app
```

8.2 Exporting classes and enums

Classes and enums export and import the same way:

```
// shapes.abe
export class Circle {
    r: i64;
    constructor(r: i64) { this.r = r; }
    area10(): i64 { return 3 * this.r * this.r; }
}

// status.abe
export enum HttpStatus { Ok = 200, NotFound = 404 }

import { Circle } from "./shapes";
import { HttpStatus } from "./status";

function main(): i64 {
    let c = new Circle(2);
    if (c.area10() != 12) return 1;
    if (HttpStatus.Ok != 200) return 2;
    return 0;
}
```

```
}
```

8.3 Sharing constants

Export module-scope **consts** to share configuration across files:

```
// config.abe
export const MAX_RETRIES: i64 = 3;
export const SERVICE_NAME: string = "abe-svc";

import { MAX_RETRIES, SERVICE_NAME } from "./config";
function main(): i64 {
    if (MAX_RETRIES != 3) return 1;
    if (SERVICE_NAME != "abe-svc") return 2;
    return 0;
}
```

Import by name (`import { a, b } from ". /m"`). Use relative paths beginning with `./`. Each name you use from a module must be **exported** there.

PART III

The Runtime Library

Abe programs link a runtime that provides strings, JSON, collections, crypto, networking, and more — callable directly from `.abe` source. This part documents that surface.

9. Strings, JSON, and `typeof`

9.1 String methods

Strings carry a rich set of methods:

```
function main(): i64 {
    let s = "Hello, World";
    if (s.length != 12)                return 1;
    if (s.toLowerCase() != "hello, world") return 2;
    if (s.toUpperCase() != "HELLO, WORLD") return 3;
    if (!s.includes("World"))          return 4;
    if (!s.startsWith("Hello"))         return 5;
    if (!s.endsWith("World"))           return 6;
    if (s.indexOf("World") != 7)        return 7;
    if (s.substring(0, 5) != "Hello")   return 8;
    if (s[0] != "H")                    return 9;    // index a character
    return 0;
}
```

Transforming and splitting:

```
function main(): i64 {
    if (" padded ".trim() != "padded")    return 1;
    if ("a-b-c".replace("-", "_") != "a_b_c") return 2;
    let parts = "x,y,z".split(",");        // ["x", "y", "z"]
    if (parts.join(" ") != "x y z")        return 3;
    return 0;
}
```

Method	Result
<code>.length</code>	character count
<code>s[i]</code>	character at index <code>i</code>
<code>.toLowerCase()</code> / <code>.toUpperCase()</code>	case-folded copy
<code>.trim()</code>	surrounding whitespace removed
<code>.includes(sub)</code> / <code>.startsWith(p)</code> / <code>.endsWith(s)</code>	<code>bool</code>
<code>.indexOf(sub)</code>	index, or <code>-1</code>
<code>.substring(start, end)</code>	substring [<code>start</code> , <code>end</code>)
<code>.replace(old, new)</code>	replace occurrences

Method	Result
<code>.split(sep)</code>	array of pieces
<code>.join(sep)</code> (on an array)	join into a string

9.2 Regular expressions

Three built-ins cover pattern matching, each taking the pattern as the first argument (POSIX extended syntax):

- `regexTest(pattern, subject) → bool`: does the pattern match anywhere?
- `regexMatch(pattern, subject) →` the first matching substring (" " if none).
- `regexReplace(pattern, subject, replacement) →` replace **every** match.

```
function main(): i64 {
    if (!regexTest("[0-9]+", "abc123")) return 1;
    if (regexTest("^[0-9]+$", "abc123")) return 2;           // anchored: no match
    let m = regexMatch("[0-9]+", "order 42 shipped");         // "42"
    if (m != "42") return 3;
    let r = regexReplace("[0-9]", "alb2c3", "#");             // "a#b#c#"
    if (r != "a#b#c#") return 4;
    return 0;
}
```

9.3 Parsing JSON

`JSON.parse` turns a JSON string into a value you read with `.field` and `[i]`:

```
function main(): i64 {
    let p = JSON.parse("{\"name\":\"abe\",\"kind\":\"lang\"}");
    if (p.name != "abe") return 1;
    if (p.kind != "lang") return 2;
    return 0;
}
```

Nested arrays and objects are indexed and accessed the same way:

```
function main(): i64 {
    let p = JSON.parse("{\"messages\":[{\"role\":\"user\",\"text\":\"hi\"}]}");
    let first = p.messages[0]; // index into a nested array
    if (first.role != "user") return 1;
    if (first.text != "hi") return 2;
    return 0;
}
```

Malformed JSON parses to a falsy value, so `if (!JSON.parse(src)) { ... }` safely detects bad input. Parsed leaf values are strings — compare and use them as text. **Not yet available:** `JSON.stringify`. Build JSON output with string concatenation or template literals for now.

9.4 `typeof` and `Array.isArray`

Use `typeof` and `Array.isArray` to validate shapes — especially on parsed JSON, where input structure isn't guaranteed:

```
function main(): i64 {
```

```

    if (typeof "x" !== "string")    return 1;
    if (typeof 42 !== "number")     return 2;
    if (typeof true !== "boolean")  return 3;
    if (!Array.isArray([1, 2, 3])) return 4;
    if (Array.isArray("nope"))      return 5;
    let p = JSON.parse("{\"tags\": [\"a\", \"b\"]}");
    if (!Array.isArray(p.tags))     return 6;    // confirm tags is really an array
    return 0;
}

```

`typeof` returns `"string"`, `"number"`, `"boolean"`, `"object"` (arrays and maps), or `"undefined"` (null/absent).

10. Crypto and Authentication

The runtime provides production-grade primitives for password storage, token issuance, and message authentication — backed by libsodium/OpenSSL.

10.1 Password hashing (Argon2id)

`passwordHash` returns a self-contained Argon2id PHC string (algorithm, parameters, salt, and tag).

`passwordVerify` checks a candidate against it in constant time. Each hash uses a fresh random salt, so the same password hashes to different strings — always store and compare the encoded hash, never the password.

```

function main(): i64 {
    let pw = "correcthorsebatterystaple";
    let enc = passwordHash(pw);           // store `enc`
    if (!passwordVerify(pw, enc)) return 1; // correct password
    if (passwordVerify("wrong", enc)) return 2; // wrong rejected
    let enc2 = passwordHash(pw);
    if (enc == enc2) return 3;             // different salt each time
    return 0;
}

```

A legacy SHA-256 verifier, `passwordVerifySha256(pw, hexHash, salt)`, exists for migrating older records — verify against it on login, then rehash with `passwordHash`.

10.2 JSON Web Tokens (HS256)

`jwtSignHS256(secret, payload)` issues a signed token; `jwtVerifyHS256(secret, token)` returns the payload on success or an **empty string** on any failure (bad signature, wrong secret, tampering, expired exp):

```

function main(): i64 {
    let secret = "super-secret-key";
    let payload = "{\"sub\":\"user-42\"}";
    let token = jwtSignHS256(secret, payload);
    let verified = jwtVerifyHS256(secret, token);
    if (verified !== payload) return 1; // round-trip
    if (jwtVerifyHS256("wrong", token).length !== 0) return 2; // wrong secret
}

```

```
    return 0;
}
```

An **exp** (expiry) claim in the payload is enforced automatically. To read a token's payload **without** verifying (e.g. to find which key to check), `jwtPayloadHS256(token)` returns the unverified payload.

10.3 HMAC, random tokens, and constant-time compare

```
function main(): i64 {
    // HMAC-SHA256, lowercase hex (RFC test vector)
    let h = hmacSha256Hex("key",
        "The quick brown fox jumps over the lazy dog");
    if (h != "f7bc83f430538424b13298e6aa6fb143ef4d59a14946175997479dbc2d1a3cd8")
        return 1;

    // Cryptographically-random hex tokens: N bytes -> 2N hex chars
    let session = randomTokenHex(16); // 32 hex chars
    if (session.length != 32) return 2;

    // Constant-time comparison – use for secrets/tokens, not `==`
    if (!secureEquals("abc", "abc")) return 3;
    if (secureEquals("abc", "abd")) return 4;
    return 0;
}
```

Function	Purpose
<code>passwordHash(pw) / passwordVerify(pw, enc)</code>	Argon2id password storage
<code>passwordVerifySha256(pw, hex, salt)</code>	legacy SHA-256 verify (migration)
<code>jwtSignHS256 / jwtVerifyHS256 / jwtPayloadHS256</code>	HS256 JWTs
<code>hmacSha256Hex(key, msg)</code>	HMAC-SHA256 → lowercase hex
<code>randomTokenHex(byteLen)</code>	secure random hex token
<code>secureEquals(a, b)</code>	constant-time string compare

11. Environment, Files, and Caching

11.1 Environment variables

`getEnv(name)` reads an OS environment variable. An unset variable returns the empty string (never null), so the result is always safe to use directly:

```
function main(): i64 {
    let greeting = getEnv("ABE_MANUAL_GREETING");
    let missing = getEnv("ABE_DEFINITELY_UNSET_XYZ"); // -> ""
    if (missing != "") return 1;
    return 0;
}
```

Combine with `=="` for defaults: `let port = getEnv("PORT"); if (port == "") { port = "8080"; }`.

11.2 Files

`writeFile(path, content)` writes a file, `readFile(path)` returns its contents, and `fileExists(path)` tests for presence:

```
function main(): i64 {
  let path = "/tmp/abe_demo.txt";
  writeFile(path, "persisted-value");
  if (!fileExists(path)) return 1;
  let content = readFile(path);
  if (content != "persisted-value") return 2;
  return 0;
}
```

`readFileSync` / `writeFileSync` are accepted as aliases of `readFile` / `writeFile`.

11.3 In-process caching

The runtime provides named in-process caches — handy for memoizing expensive results or HTTP responses without a Redis dependency. Entries are key/value strings with an optional TTL in seconds (`0` = no expiry):

```
function main(): i64 {
  httpCacheSet("users", "u:1", "Alice", 0); // cache, key, value, ttl
  let hit: string = httpCacheGet("users", "u:1");
  if (hit != "Alice") return 1;
  if (httpCacheGet("users", "u:404") != "") return 2; // miss -> ""
  let stats = httpCacheStats("users");
  if (stats.entries != "1") return 3;
  if (!httpCacheDelete("users", "u:1")) return 4; // true once
  if (httpCacheDelete("users", "u:1")) return 5; // false after
  return 0;
}
```

Function	Purpose
<code>httpCacheSet(cache, key, value, ttl)</code>	store (ttl seconds, 0 = none)
<code>httpCacheGet(cache, key)</code>	value, or "" on miss
<code>httpCacheDelete(cache, key)</code>	true if it existed
<code>httpCacheClear(cache)</code>	drop all entries
<code>httpCacheStats(cache)</code>	<code>.entries</code> / <code>.hits</code> / <code>.misses</code> / <code>.evictions</code>

Multiple named caches are independent ("`users`" and "`sessions`" don't collide).

12. HTTP

12.1 A web server

Build a server with `server_new`, register routes, and start it with `server_listen` (which blocks, serving requests). Route handlers take a request and a response (`req: ptr`, `res: ptr`) and write the response:

```
function hello(req: ptr, res: ptr): void {
    response_status(res, 200);
    response_header(res, "Content-Type", "application/json");
    response_json(res, "{\"message\": \"Hello, Abe!\"}");
}
function getUser(req: ptr, res: ptr): void {
    let id: string = request_param(req, "id");           // :id path parameter
    let page: string = request_query(req, "page");       // ?page= query parameter
    response_text(res, "user " + id);
}
function createUser(req: ptr, res: ptr): void {
    let body: string = request_body(req);               // request body (POST/PUT)
    response_status(res, 201);
    response_json(res, body);
}
function main(): i64 {
    let app = server_new();
    server_port(app, 8080);
    server_cors(app, "*");
    server_get(app, "/api/hello", hello);
    server_get(app, "/api/users/:id", getUser);         // :id binds a path parameter
    server_post(app, "/api/users", createUser);
    server_listen(app);                                // blocks, serving requests
    return 0;
}
```

Server	Request (`req`)	Response (`res`)
<code>server_new()</code>	<code>request_param(req, name) — :name</code>	<code>response_status(res, code)</code>
<code>server_port(app, n)</code>	<code>request_query(req, name) — ? name=</code>	<code>response_header(res, k, v)</code>
<code>server_cors(app, origin)</code>	<code>request_body(req)</code>	<code>response_json(res, json)</code>
<code>server_get/post/put/delete(app, path, handler)</code>	<code>request_header(req, name)</code>	<code>response_text(res, s)</code>
<code>server_use(app, mw) — middleware</code>	<code>request_method(req) / request_path(req)</code>	<code>response_html(res, html)</code>
<code>server_listen(app) — start (blocking)</code>		

Register a function value as middleware with `server_use` (it runs before route handlers and can short-circuit), and a shutdown hook with `on_shutdown`.

12.2 An HTTP client

`httpGet(url)` and `httpPost(url, body)` make requests and return a response object; read it with `httpResponseOk` and `httpResponseText`. A failed connection returns a non-OK response rather than crashing:

```
function main(): i64 {
    let resp = httpGet("https://api.example.com/v1/items");
    if (httpResponseOk(resp)) {
        let body = httpResponseText(resp);
        // ... use body ...
    }
    let created = httpPost("https://api.example.com/v1/items", "{\"name\":\"x\"}");
    return 0;
}
```

`httpPostWithHeaders(url, body, headersJson)` adds request headers (e.g. an `Authorization` bearer token) — this is what backs LLM/API integrations.

PUT/PATCH/DELETE client verbs are planned.

12.3 DNS

`dnsResolveTxtJson(domain)` performs a TXT lookup and returns a JSON string (`{"ok":bool,"records":[...]}`), so results are easy to inspect:

```
function main(): i64 {
    let result: string = dnsResolveTxtJson(""); // empty domain
    if (!result.includes("\"ok\":false")) return 1;
    return 0;
}
```

The server example above is verified **live** — the test suite boots it, sends a request to `/api/users/:id`, and checks the response — so dynamic responses built by concatenation work as shown. Broader server behaviour (middleware, streaming, graceful shutdown) is exercised by the compiler's HTTP integration test.

13. Database

The runtime includes a PostgreSQL client. Connect with `dbConnect`, run parameterized queries, and read results. Connections and queries are **null-safe**: a failed connection yields `null` and every downstream call degrades gracefully instead of crashing, so error handling is a simple truthy check.

13.1 Connecting

```
function main(): i64 {
    let db = dbConnect("postgres://user:pass@127.0.0.1:1/mydb");
    if (db) {
        // truthful only when the connection succeeds
        dbDisconnect(db);
    }
    // Offline here, dbConnect returns null and every call below stays null-safe.
    return 0;
}
```

13.2 Parameterized queries

Always pass user-supplied values as parameters (`$1`, `$2`, ...) rather than concatenating them into SQL — this is injection-safe. Because every accessor is null-safe, you can read results without a separate connection check: with no connection `dbResultRowCount` is `0` and the loop simply doesn't run.

```
function main(): i64 {
  let db = dbConnect("postgres://127.0.0.1:1/mydb");
  let params = ["alice@example.com"];
  // Parameterized query – values bind to $1, $2, ... (SQL-injection safe).
  let r = dbQueryParams(db, "SELECT id, name FROM users WHERE email = $1",
params);
  let rows = dbResultRowCount(r);    // null-safe: 0 when there is no result
  let i = 0;
  while (i < rows) {                // with a live database, iterates matches
    let name = dbResultGet(r, i, 1); // (row, column) -> string
    i = i + 1;
  }
  return 0;
}
```

`dbQuery(db, sql)` runs a query without parameters. Both the connection URL and SQL text may be built with concatenation or template literals (though user-supplied *values* belong in parameters, not the SQL string).

Function	Purpose
<code>dbConnect(url)</code>	connect; <code>null</code> on failure
<code>dbDisconnect(db)</code>	close the connection
<code>dbPing(db)</code>	health check (<code>bool</code>)
<code>dbQuery(db, sql)</code>	run a query → result, or <code>null</code>
<code>dbQueryParams(db, sql, params)</code>	parameterized query (<code>\$1...</code>)
<code>dbResultRowCount(r)</code>	number of rows (<code>0</code> if <code>null</code>)
<code>dbResultGet(r, row, col)</code>	a cell value (string)
<code>dbResultRows(r)</code>	rows as an array of <code>{column: value}</code> maps

`dbResultRows` is convenient for JSON-shaped access — `rows[0].email` instead of `dbResultGet(r, 0, 1)`.

The examples in this chapter are verified against the **null-safe contract** offline (no live database needed): a failed `dbConnect` returns `null` and every accessor degrades safely. Running real queries requires a reachable PostgreSQL server.

14. Licensing

Abe has two separately-licensed pieces:

- **abec**, the compiler — a **commercial product** of E-Tools AI Corporation, **distributed as a prebuilt binary under the Abec Compiler EULA**. The compiler source is private. Each build runs free for a 30-

day evaluation**; once the grace window expires a license key is required. Extended trials are available — contact licensing@e-tools.ai.

- **The Abe runtime** — the library your compiled programs link. It is distributed as a **freely-redistributable binary** under the **Abe Runtime Redistribution License**: you may ship it inside the native executables `abec` produces.

In other words: you license the compiler to *build* `.abe` programs, and you may *distribute* the resulting native binaries (which embed the runtime) under the runtime's redistribution terms.

The authoritative documents live alongside the distribution (in the `abec` repository and the runtime package):

Document	Covers
Abec-Compiler-EULA	terms for using the <code>abec</code> compiler
Abe-Runtime-Redistribution-License	redistributing the linked runtime in your apps
Abec-Licensing-FAQ	common licensing questions

Refer to those documents for the binding terms — this chapter is a summary, not the license itself. For questions not covered by the FAQ, contact E-Tools AI Corporation.

Appendix A — A Worked Example

This appendix ties the language and runtime together in one small program: a **task API** that serves JSON over HTTP. It uses an `enum` for domain values, a helper function, request path parameters, string-built JSON, and routing — features from across this manual in a single, runnable service.

```
// task_api.abe — a tiny JSON HTTP service
enum Priority { Low, Medium, High }

function priorityName(p: Priority): string {
    if (p == Priority.High)    return "high";
    if (p == Priority.Medium) return "medium";
    return "low";
}

function health(req: ptr, res: ptr): void {
    response_status(res, 200);
    response_json(res, "{\"status\":\"ok\",\"service\":\"tasks\"}");
}

function getTask(req: ptr, res: ptr): void {
    let id = request_param(req, "id");           // :id from the path
    let body = "{\"id\":\"" + id + "\",\"priority\":\""
        + priorityName(Priority.High) + "\"}";
    response_status(res, 200);
    response_json(res, body);
}

function main(): i64 {
```

```

    let app = server_new();
    server_port(app, 8080);
    server_cors(app, "*");
    server_get(app, "/health", health);
    server_get(app, "/tasks/:id", getTask);
    return server_listen(app);           // blocks, serving requests
}

```

Build and run it, then make requests:

```

./abec task_api.abec -o task_api
./task_api &                                # start the server
curl http://127.0.0.1:8080/health            # {"status":"ok","service":"tasks"}
curl http://127.0.0.1:8080/tasks/42         # {"id":"42","priority":"high"}

```

From here you would add `server_post` routes that read `request_body`, validate input, hash passwords with `passwordHash`, issue session tokens with `jwtSignHS256`, cache responses with `httpCacheSet`, and persist with `dbQueryParams` — each covered in Part III. The whole service compiles to a single native binary that links the runtime; there is no interpreter, package manager, or external dependency tree to deploy.

Abe Pro User Manual — Level 1 + Runtime. Level 2 (Machine Learning) ships in a later release.